

PSO a fondo

Explicación técnica completa del algoritmo — documento personal, no forma parte de la entrega.

Acompaña a `pso_pendulo_invertido.ipynb` y a `borrador_entrega.md`.

Índice

1. De dónde sale PSO, y qué problema resuelve
2. Las ecuaciones completas, término por término
3. El problema de la divergencia
4. El factor de restricción de Clerc-Kennedy — derivación
5. Exploración vs. explotación
6. Por qué esta variante y no otras
7. Nuestra implementación, línea por línea
8. Leyendo lo que encontramos, con la teoría en la mano

1. De dónde sale PSO, y qué problema resuelve

PSO (Kennedy y Eberhart, 1995) nace de un modelo de simulación social — imitar cómo se mueve una bandada de pájaros o un banco de peces buscando comida — que sus autores notaron que, con pequeños ajustes, servía para optimizar funciones. La metáfora se mantiene en el nombre y en la intuición, pero lo que importa para usarlo bien es entenderlo como lo que es: un método de **optimización estocástica basada en población, sin gradiente**.

Eso responde una pregunta concreta: ¿para qué tipo de problema sirve? Sirve cuando:

- La función de costo **no tiene forma cerrada** — no se puede escribir $f(x)$ como una expresión algebraica, solo se puede *evaluar* (acá, correr una simulación de 4 segundos).
- No hay gradiente disponible, o no es confiable — nuestra f tiene una penalización discontinua (+500 si el péndulo cae), así que ni siquiera es diferenciable en todos lados.
- El espacio de búsqueda es continuo y de dimensión moderada (acá, 3: K_p , K_i , K_d) — muy alto (cientos de dimensiones) y PSO empieza a competir en desventaja con métodos más especializados.

Contraste rápido con un Algoritmo Genético (GA): un GA mantiene una población discreta y la hace evolucionar con selección, cruza y mutación — operadores que tienen sentido claro sobre representaciones discretas (bitstrings, permutaciones). PSO, en cambio, modela un *enjambre continuo*: cada partícula se mueve por el espacio con una velocidad que se actualiza cada iteración, sin cruza ni mutación explícitas. Para un problema de sintonía de parámetros continuos como este (tres ganancias reales), PSO encaja mejor porque el espacio de búsqueda ya es continuo por naturaleza — no hay que forzar una codificación discreta sobre algo que no lo es.

2. Las ecuaciones completas, término por término

Cada partícula i tiene una posición $\vec{x}_i$ (acá, una terna (K_p, K_i, K_d)) y una velocidad $\vec{v}_i$ en ese mismo espacio. Además, el algoritmo recuerda dos cosas:

- $\vec{p}_i$ — la mejor posición que *esa partícula* encontró en toda su historia ("pbest").
- $\vec{g}$ — la mejor posición que *cualquier* partícula del enjambre encontró ("gbest").

En cada iteración, la velocidad se actualiza combinando tres tendencias:

$$\vec{v}_i \leftarrow w \vec{v}_i + \varphi_1 r_1 \odot (\vec{p}_i - \vec{x}_i) + \varphi_2 r_2 \odot (\vec{g} - \vec{x}_i)$$
$$\vec{x}_i \leftarrow \vec{x}_i + \vec{v}_i$$

TÉRMINO	QUÉ REPRESENTA	NOMBRE COMÚN
$w \vec{v}_i$	La partícula sigue moviéndose en la dirección que traía	Inercia
$\varphi_1 r_1 (\vec{p}_i - \vec{x}_i)$	Atracción hacia el mejor punto que <i>ella misma</i> ya encontró	Componente cognitivo
$\varphi_2 r_2 (\vec{g} - \vec{x}_i)$	Atracción hacia el mejor punto que <i>el enjambre</i> encontró	Componente social

r_1, r_2 son vectores de números aleatorios uniformes en $[0, 1]$, uno por dimensión, redibujados en cada iteración — son los que le dan al algoritmo su carácter estocástico y evitan que todas las partículas se muevan en perfecta sincronía. φ_1, φ_2 son constantes que pesan cuánto importa "lo que yo aprendí" contra "lo que aprendió el grupo".

La tensión de fondo: el componente cognitivo mantiene diversidad (cada partícula insiste un poco en su propio historial, así que el enjambre no colapsa instantáneamente a un solo punto) — es lo que da *exploración*. El componente social tira a todas hacia el mismo lugar — es lo que da *explotación*, refinar alrededor de lo mejor encontrado. Prácticamente todo el comportamiento macroscópico del enjambre

(converge rápido y mal, o converge lento y bien, o directamente diverge) sale de cómo se balancean estos dos términos junto con la inercia. La sección 4 muestra por qué la versión original de esta ecuación (con $w = 1$ fijo) es inestable, y cómo se arregla.

3. El problema de la divergencia

La forma original de PSO (1995) usaba $w = 1$ — sin ningún factor de amortiguación sobre la inercia. El problema: nada en la ecuación evita que $\vec{v}_i$ crezca sin límite. Si en varias iteraciones seguidas los términos cognitivo y social empujan en la misma dirección, la velocidad se acumula, la partícula empieza a dar saltos cada vez más grandes, y termina saliéndose del espacio de búsqueda relevante — "explota" en vez de converger.

La solución de la época (Eberhart y Shi, 1998) fue imponer un **V_max** — un límite duro a la velocidad, elegido a mano por el usuario. Funciona, pero es un parche: no hay una receta principada para elegir **V_max**, y el comportamiento del enjambre termina siendo sensible a ese número arbitrario.

4. El factor de constricción de Clerc-Kennedy — derivación

Clerc y Kennedy (2002) resolvieron el problema de raíz, no con un parche: reescribieron la ecuación de velocidad multiplicando *todo* por un factor χ , elegido matemáticamente para que el sistema dinámico converja por construcción, sin necesitar ningún **V_max**:

$$\vec{v}_i \leftarrow \chi [\vec{v}_i + \varphi_1 r_1 (\vec{p}_i - \vec{x}_i) + \varphi_2 r_2 (\vec{g} - \vec{x}_i)]$$

con

$$\chi = \frac{2}{\left| 2 - \varphi - \sqrt{\varphi^2 - 4\varphi} \right|}, \quad \varphi = \varphi_1 + \varphi_2 > 4$$

De dónde sale ese número

La idea es tratar el movimiento de una partícula (simplificando a una dimensión y reemplazando los números aleatorios por sus valores esperados) como un **sistema dinámico lineal de segundo orden**:

$$x_{t+1} = x_t + v_{t+1}, \quad v_{t+1} = \chi [v_t + \varphi_1 (p - x_t) + \varphi_2 (g - x_t)]$$

Esto se puede escribir como una recurrencia lineal de la forma $x_{t+1} = A x_t + B x_{t-1} + C$, cuya estabilidad (converge vs. oscila sin control vs. diverge) queda determinada por las **raíces de su ecuación característica** — exactamente el mismo tipo de análisis que se usa para estabilidad de sistemas de control discretos (polos dentro del círculo unitario). Clerc y Kennedy resolvieron esa ecuación característica en función de φ y despejaron el χ que pone las raíces justo en el borde de la estabilidad (convergencia garantizada, sin oscilación descontrolada). El resultado es la fórmula de arriba.

Con el valor estándar recomendado, $\varphi_1 = \varphi_2 = 2.05$ (por lo tanto $\varphi = 4.1$):

$$\chi = \frac{2}{\left| 2 - 4.1 - \sqrt{4.1^2 - 4 \cdot 4.1} \right|} \approx 0.7298$$

Ese $\chi \approx 0.73$ es, en la práctica, un **freno**: cada iteración, la velocidad resultante se multiplica por un factor menor a 1, así que cualquier acumulación descontrolada se amortigua sola. Es matemáticamente equivalente a un esquema de inercia decreciente, pero derivado de un análisis de estabilidad, no elegido a ojo.

Por qué esto importa más allá de "funciona mejor": con V_{max} a mano, si el enjambre diverge, la respuesta es "probar otro V_{max} " — no hay garantía previa. Con constricción, la convergencia está *demostrada* antes de correr una sola iteración, para cualquier φ_1, φ_2 que cumplan $\varphi > 4$. Es la diferencia entre ajustar un hiperparámetro por ensayo y error, y elegir uno con una condición matemática que lo respalda.

5. Exploración vs. explotación, revisitada con χ en la ecuación

χ no solo estabiliza — también fija cuánto "encoge" el enjambre por iteración. Con $\varphi = 4.1$ fijo, χ queda determinado (no es un grado de libertad independiente); lo que sí se puede ajustar es la relación entre φ_1 y φ_2 (repartir el peso entre "confiar en lo propio" y "confiar en el grupo") sin tocar χ . Usar $\varphi_1 = \varphi_2$ (como en este proyecto) es la elección simétrica estándar: ninguna partícula confía más en su propio historial que en el del enjambre, o viceversa.

Un enjambre con constricción típicamente muestra dos fases visibles en el gráfico de convergencia:

1. **Exploración inicial** — las partículas parten de posiciones aleatorias por todo el espacio de búsqueda, y el mejor costo global cae rápido a medida que se descubren regiones mejores.

2. Explotación / refinamiento — una vez que el enjambre converge hacia la vecindad de $\vec{g}$, sigue moviéndose pero cada vez con pasos más chicos (por la constricción), afinando el óptimo local.

En nuestra corrida esto se ve de forma extrema: la convergencia ocurre en ~ 7 generaciones y después el costo queda perfectamente plano. Eso *no* es señal de un enjambre pobre — es consecuencia de la forma particular de nuestro costo (sección 8): con un escenario de diseño que tiene una penalización fija inevitable, gran parte del "paisaje" de costo es casi plano una vez que el resto de los escenarios ya se resolvieron bien, así que no hay mucho margen adicional que explotar.

6. Por qué esta variante y no otras

VARIANTE	POR QUÉ NO
PSO con V_{max} fijo a mano (original, 1995)	Sin garantía de convergencia; agrega un hiperparámetro sin receta clara para elegirlo.
PSO con peso de inercia decreciente linealmente (Shi-Eberhart, 1998)	Funciona bien en la práctica, pero el cronograma de decrecimiento (de qué w a qué w , en cuántas iteraciones) es otro conjunto de decisiones arbitrarias — la constricción resuelve el mismo problema con una única constante derivada matemáticamente.
Algoritmo Genético (GA)	Espacio de búsqueda ya continuo (tres ganancias reales) — codificarlo como genes discretos y diseñar cruza/mutación sobre esa codificación agrega complejidad sin necesidad.
Búsqueda Tabú (TS)	Pensada para espacios combinatorios discretos con estructura de vecindad clara (ej. rutas, asignaciones); no tiene una noción natural de "vecindad" en $\mathbb{R}^3$ continuo tan directa como PSO.

PSO con constricción de Clerc-Kennedy es, para este problema específico, la opción que necesita menos decisiones de diseño arbitrarias (no hay que elegir tasa de mutación, tamaño de torneo, cronograma de temperatura, ni `V_max`) y cuya convergencia está respaldada por análisis, no solo por observación empírica.

7. Nuestra implementación, línea por línea

```
phi1, phi2 = 2.05, 2.05
phi = phi1 + phi2
chi = 2 / abs(2 - phi - np.sqrt(phi**2 - 4 * phi))    # ec. de la sección 4

posiciones = rng.uniform(lim_inf, lim_sup, size=(n_particulas, 3))  # x_i iniciales, al azar
velocidades = rng.uniform(-1, 1, size=(n_particulas, 3)) * (lim_sup - lim_inf) * 0.1 # v_i chicas al
azar

# ... por generación:
r1 = rng.uniform(0, 1, size=(n_particulas, 3))    # un r1 por partícula, por dimension
r2 = rng.uniform(0, 1, size=(n_particulas, 3))

velocidades = chi * (
    velocidades
    + phi1 * r1 * (pbest_pos - posiciones)          # componente cognitivo
    + phi2 * r2 * (gbest_pos - posiciones)          # componente social
)
posiciones = np.clip(posiciones + velocidades, lim_inf, lim_sup)    # actualizar x, y no salir de los
límites
```

Dos detalles de implementación que no están en la ecuación "de libro" pero son necesarios en la práctica:

- `np.clip(..., lim_inf, lim_sup)` — la ecuación de Clerc-Kennedy garantiza que el sistema *converge*, no que se quede dentro de límites físicos arbitrarios (acá, $K_p/K_i/K_d$ no pueden ser negativos). El `clip` es una restricción de caja adicional, ortogonal a la estabilidad matemática de la ecuación de velocidad.
- `costos = np.array([costo(*p) for p in posiciones])` — cada evaluación de `costo()` corre 9 simulaciones completas de 4 segundos (una por escenario). Con 25 partículas y 40 generaciones, son $25 \times 40 \times 9 = 9000$ simulaciones por corrida completa del PSO — el costo computacional real está ahí, no en la actualización de posición/velocidad en sí (que es álgebra vectorial trivial).

8. Leyendo lo que encontramos, con la teoría en la mano

8.1 Por qué la convergencia es tan rápida y después perfectamente plana

El costo tiene una penalización fija de 500 para cualquier escenario que termine en caída. Con el escenario más exigente (3.0° , $+0.15$ rad/s) siendo físicamente irrecuperable (ver `borrador_entrega.md`, sección 4.7), **ninguna combinación de ganancias evita esa penalización** — es un piso de costo que ningún punto del espacio de búsqueda puede bajar. Una vez que el enjambre encuentra ganancias que resuelven bien los otros 8 escenarios, el margen de mejora que queda es chico, y con $\chi \approx 0.73$ frenando la velocidad en cada iteración, el enjambre converge a ese entorno y se queda ahí — de ahí la meseta.

8.2 Por qué K_p se pega al límite de búsqueda y K_i/K_d no

Esto no es un fenómeno del algoritmo de optimización — es la *función de costo* la que no tiene mínimo interior en la dirección de K_p (crece el error de forma monótona decreciente al subir K_p , sin nunca subir de nuevo, dentro del rango explorado). PSO está haciendo exactamente lo que se le pidió: encontrar el mínimo de f . Que ese mínimo esté en el borde del dominio de búsqueda es información sobre f , no sobre PSO — es la razón por la que, en la sección 4.6 del borrador de entrega, la conclusión es que el costo está incompleto (le falta un término que penalice conmutación), no que el algoritmo haya fallado en encontrarlo.

8.3 Qué pasaría con más partículas o más generaciones

Dado que el problema no es de exploración insuficiente (el enjambre converge en ~ 7 de 40 generaciones disponibles, con margen de sobra) sino de estructura del costo, más partículas o más generaciones no van a cambiar ni el valor de K_p en el borde ni el escenario 9 sin resolver — el óptimo encontrado ya es, esencialmente, el óptimo global de esta función de costo particular, no un óptimo local por falta de exploración. Esto se puede verificar corriendo `pso_optimizar` con semillas distintas: si el resultado (K_p en el límite, K_i/K_d interiores, escenario 9 sin resolver) se repite, confirma que es una propiedad de f , no del muestreo aleatorio de una corrida particular.

Documento personal de estudio — no forma parte de la entrega formal del Desafío Práctico (Algoritmos Evolutivos I).
Referencias: Kennedy & Eberhart (1995), "Particle Swarm Optimization"; Shi & Eberhart (1998), "A modified particle swarm optimizer"; Clerc & Kennedy (2002), "The particle swarm - explosion, stability, and convergence in a multidimensional complex space".